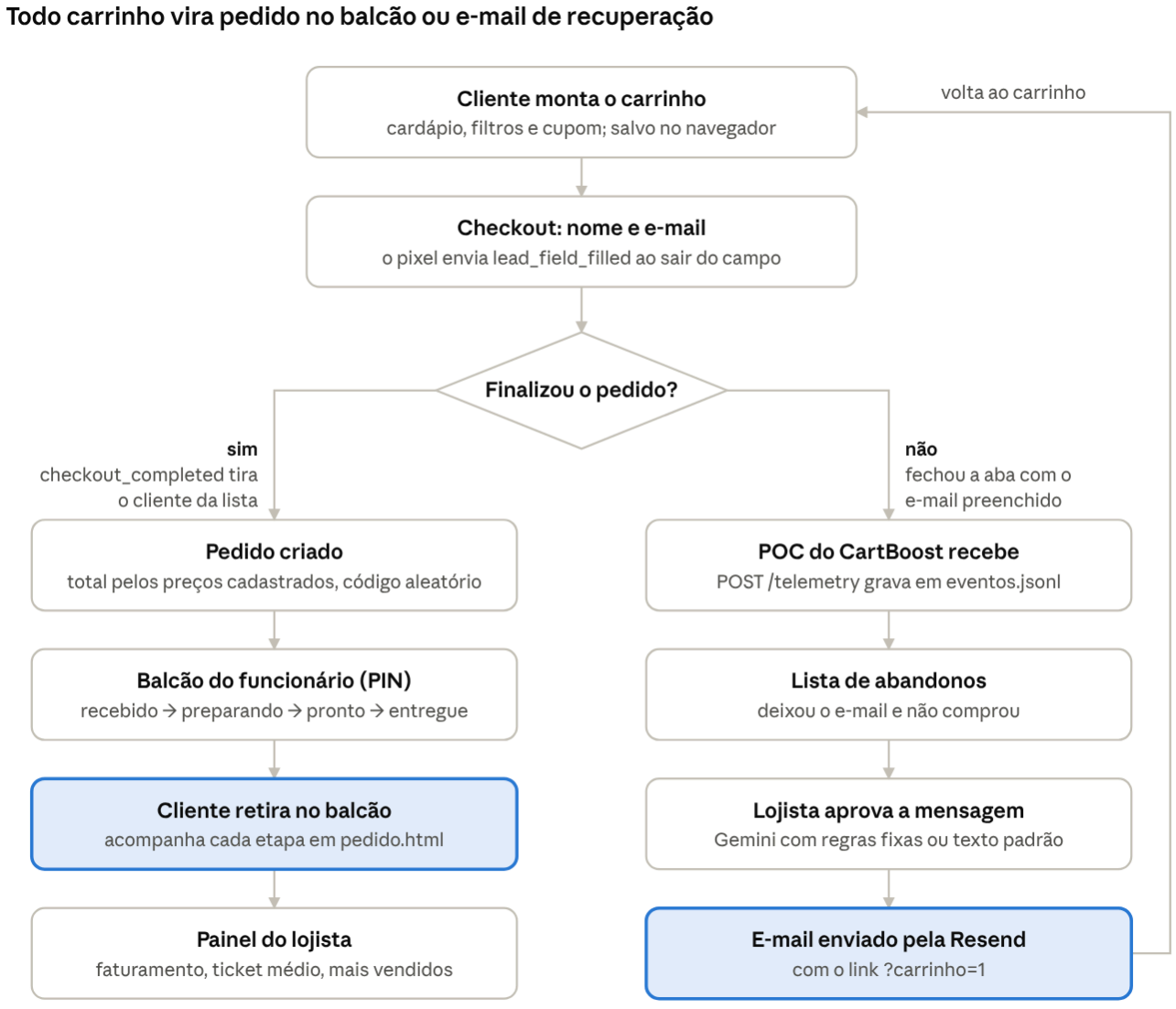
Coffe Peace + CartBoost — Pipeline do produto

Oct 14, 2026 · @Agnaldo Juarez Pereira

A Coffe Peace é uma cafeteria de demonstração com três pontas (cliente, balcão e lojista) que serve de loja de teste para o CartBoost, que recupera carrinhos abandonados por e-mail aprovado pelo lojista.

Fluxograma



fluxo do pedido e da recuperação · 2 caminhos, 1 decisão

Os dois caminhos terminam nos quadros destacados: a retirada no balcão ou o e-mail de recuperação, que leva o cliente de volta ao carrinho guardado.

As três pontas

Cada papel tem a própria página e todas leem e escrevem a mesma camada de dados (`js/dados.js`), então uma mudança numa aba aparece nas outras na hora.

Papel	Página	Como entra	O que faz
Cliente	`index.html`	Aberto, sem login	Filtra o cardápio (cafés, chás, bolos, salgados, combos), busca, monta o carrinho, aplica cupom e finaliza com nome e e-mail
Cliente	`pedido.html?c=<código>`	Link recebido ao finalizar	Acompanha o status do pedido: recebido, preparando, pronto, entregue
Funcionário	`balcao.html`	PIN único (padrão 1234)	Vê os pedidos chegando num quadro de 4 colunas, avança o status, cancela com dois cliques e marca produto como esgotado
Lojista	`lojista.html`	Botão de entrada da demo	Resumo do dia (pedidos, faturamento, ticket médio, mais vendidos), pedidos, cardápio (preço, descrição, esgotado, novo produto), cupons e ajustes (troca do PIN, restaurar a demo)

O PIN fica sem criptografia de propósito: é uma demonstração sem dados reais.

Pipeline etapa a etapa

São dois caminhos que se cruzam no carrinho: o do pedido, que termina no balcão, e o da recuperação, que traz de volta quem saiu sem comprar.

Caminho do pedido

1. O cardápio carrega os produtos da camada de dados e desenha os cards; esgotados aparecem com o botão desligado.
2. O cliente adiciona itens; o carrinho guarda quantidades e cupom no `localStorage` (chave `carrinho-v2`) e sobrevive a recarregar a página.
3. No checkout, o cliente informa nome e e-mail; a retirada é no balcão (sem entrega na branch do CartBoost).

4. `criarPedido` recalcula o total a partir dos preços cadastrados, nunca do carrinho, valida cupom e quantidades e gera um número sequencial mais um código aleatório de 16 caracteres.
5. O pedido aparece no balcão, destacado como novo; o funcionário avança: recebido → preparando → pronto → entregue (ou cancelado antes de ficar pronto).
6. O cliente acompanha cada mudança em `pedido.html`, que consulta a cada 4 segundos e também escuta as outras abas.
7. O lojista vê o pedido somar nas métricas do dia.

Caminho da recuperação (CartBoost)

1. O pixel registra o produto adicionado (atributos `data-cb-produto` e `data-cb-categoria` no botão).
2. Ao sair do campo de e-mail do checkout, o pixel envia o `lead_field_filled`.
3. Se o cliente fecha a aba sem finalizar, ele entra na lista de abandonos da POC.
4. O lojista abre o painel da POC, gera a mensagem (Gemini com regras fixas, ou a mensagem padrão), revisa e aprova.
5. O e-mail sai pela Resend com o link `?carrinho=1`, que reabre o carrinho guardado na loja.
6. Se o cliente finaliza, a loja dispara `checkout_completed` e ele sai da lista: quem comprou não recebe mensagem.

Técnicas por camada

Tudo roda sem build: HTML, CSS e JavaScript puros com módulos ES, publicados como arquivos estáticos na Vercel.

Camada	Técnica	Onde
Front-end	Módulos ES nativos (`import` / `export`), sem bundler	`script.js` , `js/*.js`
Front-end	Variáveis CSS, fonte raiz fluida com `clamp()` , largura máxima de 1800 px e margens proporcionais	`style.css`
Front-end	Layout responsivo com grid e flex; filtros que quebram linha no celular	`style.css` , `painel.css`
Front-end	Busca sem acento (`normalize("NFD")`), filtro por categoria, imagens com `loading="lazy"`	`script.js`

Camada	Técnica	Onde
Front-end	Acessibilidade: `aria-pressed` nos filtros, textos `sr-only`, `dialog` nativo na confirmação	`index.html`
Front-end	Toast em vez de `alert()`, cancelar com dois cliques em vez de `confirm()`	`js/ui.js`
Dados	Uma API única de dados (carregar, criar pedido, avançar status, métricas…), trocada por arquivo entre a demo e o banco	`js/dados.js`
Dados	Migração do estado salvo: produtos novos entram sem apagar o que o lojista mudou	`js/dados.js`
Tempo real	Evento `storage` do navegador avisa as outras abas; polling de 4 s no acompanhamento	`js/dados.js`, `js/pedido.js`
Regras	Máquina de estados do pedido com transições permitidas	`js/dados.js`
Regras	Total recalculado pelos preços cadastrados; código de acompanhamento aleatório (`crypto.randomUUID`)	`js/dados.js`
Integração	Pixel de uma linha com `defer`, atributos `data-cb-*` e uma chamada `CartBoost.track`	`index.html`, `script.js`
Integração	`navigator.sendBeacon` com `text/plain`, sem consulta prévia de CORS	POC: `public/pixel.js`
Banco	Postgres no Supabase: `CHECK`, `numeric` para dinheiro, `timestamptz`, tabela de linha única, RLS e funções `security definer`	branch `supabase-cartboost`
IA	Gemini com regras fixas, contexto tratado como dado, conferência da resposta e mensagem padrão	POC: `src/mensagem.js`
IA	Rede neural em TensorFlow.js no navegador para ranquear produtos	POC: `public/recomendacao.js`
Testes	Chrome headless (`--dump-dom`, captura de tela) com páginas de vários iframes simulando abas	local

Dados e banco

A main guarda tudo no navegador para qualquer pessoa testar sem servidor; a branch `supabase-cartboost` troca só o arquivo de dados por um banco Postgres real, mantendo as mesmas funções.

	main (publicada)	branch `supabase-cartboost` (testes)
Onde ficam os dados	`localStorage` do navegador (chave `coffe-peace-demo`)	Supabase (Postgres)
Recebimento	Retirada ou entrega	Só retirada no balcão
Contato do cliente	Nome	Nome e e-mail
CartBoost	Sem pixel	Pixel ativo

Tabelas já criadas no Supabase (escritas à mão no SQL Editor, como exercício de SQL):

- `clientes`: nome de 2 a 60 letras, e-mail único em minúsculas e restrito a domínios reservados para teste (`example.com`, `.test`), consentimento com data (`aceita_contato` só vale com `consentimento_em`) e `anonymous_id` para ligar a recuperação à compra.
- `produtos`: id em formato slug, preço `numeric(10,2)` entre 0 e 1000, categoria, disponibilidade e ordem.
- `cupons`: código em maiúsculas, desconto de até 50%, ativo ou não.
- `configuracao`: tabela de linha única (`check (id = 1)`) com o PIN do balcão.

Planejado: `pedidos` e `itens_pedido` (preço do momento da compra e horário de cada etapa), políticas RLS, funções do balcão protegidas pelo PIN, métricas com tempo médio de preparo e dados fictícios de exemplo. A categoria de `produtos` ainda aceita só `cafe` e `cha`: os bolos, salgados e combos pedem um `ALTER TABLE`.

Nenhum dado real entra no banco: Coffe Peace e a POC são vitrine; o CartBoost com clientes reais vira um SaaS à parte.

CartBoost na loja

A loja instala o CartBoost com uma tag e uma chamada; o resto o pixel descobre no HTML que já existe.

```
<script src="http://localhost:3000/pixel.js" defer></script>
```

```
window.CartBoost?.track("checkout_completed", { pedido, valor, itens, cupom
});
```

O que o pixel precisa	Como descobre na Coffe Peace
Para onde enviar	O próprio `src` da tag
Produto de interesse	`data-cb-produto` e `data-cb-categoria` nos botões de adicionar
E-mail	O campo `type="email"` do checkout, com aviso de consentimento ao lado
Compra	A chamada `checkout_completed` ao confirmar o pedido

A POC (Fastify na porta 3000) recebe os eventos em `POST /telemetry`, grava em `dados/eventos.jsonl` e lista em `GET /abandonos` quem deixou o e-mail e não comprou. No painel, o lojista gera a mensagem com a Gemini, sob regras fixas (no máximo 3 frases, sem desconto, preço ou dado pessoal no prompt), e aprova o envio pela Resend. Na POC, todo e-mail vai para o endereço de demonstração do `.env`, nunca para o e-mail digitado na loja.

O `.env` da POC liga a loja com três variáveis: `CORS_ORIGENS=http://localhost:5500`, `CATALOGO=catalogo-coffe-peace.json` e `LOJA_URL=http://localhost:5500/?carrinho=1`.

Imagens do cardápio

Todas as 34 fotos seguem o mesmo cenário (mesa redonda de madeira, janela desfocada, luz quente) e o mesmo formato: 1200×655 px, PNG com paleta de 256 cores, cerca de 300 a 550 KB.

Itens	Técnica
Cafés, Earl Grey, bolos e salgados	Geradas no site do Gemini, a partir de uma foto de referência da mesma cena
6 chás novos	Edição da foto do Earl Grey em Python (Pillow + NumPy): máscara elíptica no líquido, nova cor mantendo reflexos e sombras, e o texto da etiqueta apagado por difusão

Itens	Técnica
Combo Manhã e Combo Tarde	Montagem sobre a foto do Combo Clássico: a fatia de chocolate trocada pela de laranja e pela de cenoura, com recorte por cor, borda suave e o pão de queijo da frente protegido
12 combos sorteados	Script de sorteio (salgado + bolo + bebida, bebida + bolo, bebida + salgado) que monta painéis lado a lado, com zoom ajustado por foto para xícaras e pratos ficarem no mesmo tamanho

O preço de cada combo sorteado é a soma dos itens menos cerca de 12%, arredondada para baixo.

Testes e o que falta

Os fluxos da main foram verificados de ponta a ponta no Chrome headless, com uma página de quatro iframes fazendo o papel de quatro abas: loja, acompanhamento, balcão e lojista.

- PIN do balcão, troca do PIN e restauração da demo.
- Pedido criado na loja aparecendo no balcão; status avançando até o acompanhamento; métricas e cupons no lojista.
- Esgotado no balcão desligando o botão na loja; edição de preço refletindo no cardápio.
- Cardápio com 34 itens e etiquetas certas; filtros no celular (390 px); produtos novos entrando num estado salvo antigo sem perder as mudanças do lojista.
- Na branch: campo de e-mail, eventos do pixel (com um pixel falso) e o link `?carrinho=1` reabrindo o carrinho.
- Na POC principal do CartBoost: abandono gerando mensagem e compra cancelando a janela de recuperação.

O que falta

- [ ] Teste ponta a ponta no navegador real com a POC: os eventos da loja ainda não chegaram (suspeitas: bloqueador de anúncios ou porta 3000 ocupada).
- [ ] Commitar a branch `supabase-cartboost` e trazer para ela os chás, bolos, salgados e combos da main.
- [ ] Atualizar o catálogo da POC com os produtos novos.
- [ ] SQL: `ALTER TABLE` da categoria, `pedidos`, `itens_pedido`, RLS, funções e dados fictícios.
- [ ] Trocar o `js/dados.js` da branch pela versão que fala com o Supabase.